

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 6 Transacciones y Control de Concurrency

Alumno: Tomás Rodeghiero
Año: 2026

Resolución basada en el Teórico 8 (Transacciones y Control de Concurrency) y en experimentación sobre MySQL y PostgreSQL.

Índice

1. Introducción	2
2. Ejercicio 1: Planificaciones concurrentes y serializabilidad	2
3. Ejercicio 2: Protocolo de bloqueo de dos fases puro	4
4. Ejercicio 3: Protocolo de marcas temporales	6
5. Ejercicio 4: UPDATE y ROLLBACK en MySQL	7
6. Ejercicio 5: Dos sesiones MySQL en SERIALIZABLE	8
7. Ejercicio 6: Lectura no repetible	10
8. Ejercicio 7: Deferrable Constraints en PostgreSQL	12
9. Ejercicio 8: MVCC en PostgreSQL (xmin / xmax)	14
Conclusiones	16

1. Introducción

El Práctico 6 cierra el cuatrimestre con los conceptos sobre los que se apoya cualquier base de datos multiusuario: **transacciones** y **control de concurrencia**. Los ocho ejercicios se reparten en dos mitades complementarias:

- **Ejercicios 1–3:** parte teórica. Planificaciones concurrentes, serializabilidad por conflicto, protocolo de dos fases y protocolo de marcas temporales.
- **Ejercicios 4–8:** parte práctica. Pruebas reales en MySQL (transacciones, niveles de aislamiento, lectura no repetible) y en PostgreSQL (constraints deferibles, MVCC con `xmin/xmax`).

Cada ejercicio sigue siempre la misma estructura: **Consigna** (texto literal del PDF), **Idea** (lectura natural y concepto detrás), **Solución** (schedule, grafo, secuencia 2PL, tabla de marcas o SQL transaccional según corresponda), **Por qué funciona / por qué se rompe** (lógica de la serializabilidad o de la anomalía) y **En MySQL 8+** (equivalente en InnoDB cuando aplica: niveles de aislamiento, `FOR UPDATE`, MVCC, `information_schema.INNODB_TRX`, limitaciones específicas como la ausencia de FKs deferibles).

Notación. A lo largo del documento se usa la notación corta $r_i(Q)$ para “ T_i lee Q ” y $w_i(Q)$ para “ T_i escribe Q ”. Las asignaciones locales (por ejemplo $X := X - N$) no generan conflictos entre transacciones y se omiten en el análisis. Los bloqueos se denotan $LX_i(Q)$ (exclusivo) y $LC_i(Q)$ (compartido), y $UX_i(Q)$ / $UC_i(Q)$ para sus desbloques.

2. Ejercicio 1: Planificaciones concurrentes y serializabilidad

Consigna

Dadas las siguientes transacciones:

- T_1 : leer(X); $X := X - N$; escribir(X); leer(Y); $Y := Y + N$; escribir(Y);
- T_2 : leer(X); $X := X + M$; escribir(X);

Dar dos planificaciones concurrentes, una serializable en cuanto a conflicto y la otra no, justifique su respuesta.

Idea

Dos instrucciones $I_i \in T_i$ e $I_j \in T_j$ están en *conflicto* si acceden al mismo ítem Q y al menos una es escritura (pares $r-w$, $w-r$, $w-w$). Una planificación es *serializable por conflicto* si su grafo de precedencia (un nodo por transacción, arco $T_i \rightarrow T_j$ cuando una operación de T_i precede en el schedule a una operación conflictiva de T_j) es acíclico.

Solución

Planificación 1: serializable por conflicto.

$$S_1 = r_1(X), w_1(X), r_2(X), w_2(X), r_1(Y), w_1(Y)$$

Paso	Transacción	Operación
1	T_1	leer(X)
2	T_1	$X := X - N$
3	T_1	escribir(X)
4	T_2	leer(X)
5	T_2	$X := X + M$
6	T_2	escribir(X)
7	T_1	leer(Y)
8	T_1	$Y := Y + N$
9	T_1	escribir(Y)

Conflictos sobre X (a Y accede sólo T_1):

- $w_1(X)$ antes de $r_2(X) \Rightarrow T_1 \rightarrow T_2$.
- $w_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.
- $r_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.

Grafo de precedencia:



Sin ciclos: S_1 es **serializable por conflicto**, equivalente a T_1 seguida de T_2 .

Planificación 2: no serializable por conflicto.

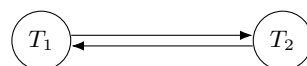
$$S_2 = r_1(X), r_2(X), w_1(X), w_2(X), r_1(Y), w_1(Y)$$

Paso	Transacción	Operación
1	T_1	leer(X)
2	T_2	leer(X)
3	T_1	$X := X - N$
4	T_1	escribir(X)
5	T_2	$X := X + M$
6	T_2	escribir(X)
7	T_1	leer(Y)
8	T_1	$Y := Y + N$
9	T_1	escribir(Y)

Conflictos:

- $r_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.
- $r_2(X)$ antes de $w_1(X) \Rightarrow T_2 \rightarrow T_1$.
- $w_1(X)$ antes de $w_2(X) \Rightarrow T_1 \rightarrow T_2$.

Grafo de precedencia:



Hay ciclo $T_1 \leftrightarrow T_2$: S_2 **no es serializable por conflicto**.

Por qué funciona / por qué se rompe

- En S_1 , T_1 termina su trabajo sobre X antes de que T_2 empiece, así que T_2 ve el efecto de T_1 y produce un valor coherente. El orden serial equivalente es $T_1; T_2$.
- En S_2 , ambas transacciones leen el X *original* (porque sus lecturas anteceden a las escrituras de la otra), y luego cada una sobrescribe con un valor que no refleja el efecto de la otra: clásico *lost update*.
- El ciclo en el grafo es el síntoma formal: cada transacción quedó “antes” de la otra simultáneamente, lo que no se puede materializar en ningún orden serial.

En MySQL 8+

Concepto teórico. Aun así, los dos schedules se pueden *materializar* en MySQL/InnoDB para ver cómo el motor reacciona:

```
-- En SERIALIZABLE, InnoDB toma S-lock implícito en cada SELECT.
-- El S2 problematico no se puede reproducir tal cual: cuando T2 quiera
-- hacer r2(X) despues de w1(X), InnoDB la bloquea hasta que T1 confirme.
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION; -- T1
SELECT X FROM tab WHERE id = 1; -- toma S-lock
UPDATE tab SET X = X - N WHERE id = 1; -- promueve a X-lock

-- Sesión 2 (T2)
START TRANSACTION;
SELECT X FROM tab WHERE id = 1; -- queda esperando: hay X-lock de T1
```

Diferencia clave con el modelo teórico: InnoDB no permite arbitrariamente cualquier intercalado; los locks fuerzan que los schedules ejecutados sean serializables. Para reproducir S_2 haría falta bajar el nivel a READ COMMITTED y aceptar la posibilidad de *lost update*.

3. Ejercicio 2: Protocolo de bloqueo de dos fases puro

Consigna

Dadas las transacciones del ejercicio anterior, dar una planificación para su ejecución concurrente utilizando el protocolo de dos fases puro.

Idea

El *Protocolo de Bloqueo de Dos Fases* (2PL) divide la vida de toda transacción en dos etapas: *crecimiento* (solo se piden bloqueos) y *decrecimiento* (solo se liberan). 2PL garantiza schedules serializables por conflicto. El protocolo *puro* permite liberar bloqueos antes del *commit*, a diferencia del *estricto* (los retiene hasta el *commit*) y el *riguroso* (también difiere lecturas hasta el *commit*).

Solución

Como ambas transacciones escriben sobre X , hace falta bloqueo *exclusivo* (LX) sobre X para las dos; T_1 además bloquea Y porque también lo escribe.

Paso	Transacción	Operación
1	T_1	$LX_1(X)$
2	T_1	$LX_1(Y)$
3	T_1	$\text{leer}(X)$
4	T_1	$X := X - N$
5	T_1	$\text{escribir}(X)$
6	T_2	$LX_2(X)$ ($\rightarrow$ espera)
7	T_1	$UX_1(X)$ (inicia decrecimiento de T_1)
8	T_2	$LX_2(X)$ (concedido)
9	T_2	$\text{leer}(X)$
10	T_1	$\text{leer}(Y)$
11	T_2	$X := X + M$
12	T_1	$Y := Y + N$
13	T_2	$\text{escribir}(X)$
14	T_2	$UX_2(X)$
15	T_2	commit
16	T_1	$\text{escribir}(Y)$
17	T_1	$UX_1(Y)$
18	T_1	commit

Verificación de las fases:

T_1 . Crecimiento: pasos 1–2 (adquiere X y Y). Punto de viraje: paso 7 (libera X). Decrecimiento: del paso 7 en adelante; opera con los bloqueos que tenía y libera Y al final (paso 17).

T_2 . Crecimiento: paso 8 (consigue X). Decrecimiento: paso 14 (libera X). No vuelve a pedir bloqueos.

Por qué funciona / por qué se rompe

- Cada transacción tiene un único punto donde pasa de crecer a decrecer: es 2PL *puro*.
- La ejecución es concurrente (los pasos 8–14 entrelazan operaciones de T_2 con T_1) pero el orden efectivo equivalente a sobre X es serial $T_1 \rightarrow T_2$.
- 2PL puro no exige retener los bloqueos hasta el **commit**: $UX_1(X)$ ocurre en el paso 7, antes del commit de T_1 . Eso *permite* la concurrencia pero abre la puerta a rollback en cascada si T_2 leyera escrituras no confirmadas; la variante *estricta/rigurosa* cierra ese problema reteniendo los bloqueos hasta el commit.

En MySQL 8+

InnoDB *no* usa 2PL puro: implementa *2PL estricto* (**strict 2PL**) — los bloqueos exclusivos se retienen hasta el COMMIT/ROLLBACK. Por eso un esquema como el de arriba (donde T_1 libera X en el paso 7 *sin haber commiteado*) no es expresable directamente en MySQL. Para ver cómo InnoDB serializa los accesos:

```
-- Sesión 1 (T1)
SET autocommit = 0;
START TRANSACTION;
SELECT X FROM tab WHERE id=1 FOR UPDATE;  -- toma X-lock, lo retiene hasta commit
UPDATE tab SET X = X - 10 WHERE id=1;

-- Sesión 2 (T2): bloquea hasta que sesión 1 haga COMMIT
START TRANSACTION;
SELECT X FROM tab WHERE id=1 FOR UPDATE;  -- queda esperando
```

Diferencia clave: en InnoDB no se puede observar la fase de decrecimiento “antes del commit” que sí permite 2PL puro. A cambio, se evita por construcción el rollback en cascada (porque nadie lee escrituras no confirmadas a través de X-locks).

4. Ejercicio 3: Protocolo de marcas temporales

Consigna

Dadas las siguientes transacciones, dar una planificación utilizando el protocolo de marcas temporales, mostrar cómo varían $MT-E$, $MT-L$ de X e Y .

- T_1 : leer(X); escribir(X); leer(Y); escribir(Y);
- T_2 : leer(X); escribir(X);
- T_3 : leer(Y); escribir(Y);

Idea

A cada transacción T_i se le asigna una marca $MT(T_i)$ que la ordena (más chica = más vieja). Cada ítem Q guarda $MT-E(Q)$ (mayor marca de quienes lo escribieron) y $MT-L(Q)$ (mayor marca de quienes lo leyeron). Las reglas:

T_i ejecuta $r(Q)$. Si $MT(T_i) < MT-E(Q)$: rechazo y rollback. Si no: ejecuta y $MT-L(Q) \leftarrow \max(MT-L(Q), MT(T_i))$.

T_i ejecuta $w(Q)$. Si $MT(T_i) < MT-L(Q)$ o $MT(T_i) < MT-E(Q)$: rechazo y rollback. Si no: ejecuta y $MT-E(Q) \leftarrow MT(T_i)$.

Solución

Asignación inicial: $MT(T_1) = 1$, $MT(T_2) = 2$, $MT(T_3) = 3$. Estado inicial: $MT-E(X) = MT-L(X) = 0$, $MT-E(Y) = MT-L(Y) = 0$.

Schedule propuesto:

$$r_1(X), w_1(X), r_2(X), w_2(X), r_1(Y), w_1(Y), r_3(Y), w_3(Y)$$

Paso	Operación	Validación	$MT-E(X)$	$MT-L(X)$	$MT-E(Y)$	$MT-L(Y)$
0	inicio	—				
1	$r_1(X)$	$1 \geq MT-E(X)=0 \Rightarrow \text{OK}; MT-L(X)=\max(0,1)=1$		0		
2	$w_1(X)$	$1 < MT-L(X)=1? \text{ no}; 1 < MT-E(X)=0? \text{ no} \Rightarrow \text{OK}; MT-E(X)=1$	1			
3	$r_2(X)$	$2 \geq MT-E(X)=1 \Rightarrow \text{OK}; MT-L(X)=\max(1,2)=2$		1		
4	$w_2(X)$	$2 < MT-L(X)=2? \text{ no}; 2 < MT-E(X)=1? \text{ no} \Rightarrow \text{OK}; MT-E(X)=2$	2			
5	$r_1(Y)$	$1 \geq MT-E(Y)=0 \Rightarrow \text{OK}; MT-L(Y)=\max(0,1)=1$			0	0
6	$w_1(Y)$	$1 < MT-L(Y)=1? \text{ no}; 1 < MT-E(Y)=0? \text{ no} \Rightarrow \text{OK}; MT-E(Y)=1$			1	
7	$r_3(Y)$	$3 \geq MT-E(Y)=1 \Rightarrow \text{OK}; MT-L(Y)=\max(1,3)=3$			2	1
8	$w_3(Y)$	$3 < MT-L(Y)=3? \text{ no}; 3 < MT-E(Y)=1? \text{ no} \Rightarrow \text{OK}; MT-E(Y)=3$			3	2

Resultado final. Sin rollbacks. X : $MT-E(X) = 2$, $MT-L(X) = 2$. Y : $MT-E(Y) = 3$, $MT-L(Y) = 3$.

Por qué funciona / por qué se rompe

- El orden efectivo de conflictos respeta el orden temporal de las marcas: T_1 aparece antes que T_2 en los conflictos sobre X , y T_1 antes que T_3 en los conflictos sobre Y .

- Como no hay accesos cruzados (ninguna T_i con MT chica intenta operar sobre un ítem ya tomado por una con MT mayor), no se viola ninguna condición de retroceso.
- Si el schedule hubiese sido $r_2(X), w_2(X), w_1(X)$, en el paso 3 T_1 querría escribir un valor obsoleto ($MT(T_1) = 1 < MT-E(X) = 2$): se rechazaría y T_1 tendría que reiniciar con una marca nueva.

En MySQL 8+

Concepto teórico. InnoDB *no* implementa ordenación por marcas temporales: su mecanismo de concurrencia es 2PL estricto con detección automática de deadlocks (no usa marcas para abortar, sino que detecta ciclos en el grafo *wait-for*). Por lo tanto este ejercicio no tiene reproducción directa en MySQL; lo más cercano sería usar `innodb_deadlock_detect` para observar cómo el motor decide qué transacción abortar cuando hay un ciclo de bloqueos.

```
-- Inspeccion de transacciones activas y sus locks
SELECT * FROM information_schema.INNODB_TRX;

-- Activar/desactivar deteccion de deadlock
SHOW VARIABLES LIKE 'innodb_deadlock_detect';
```

5. Ejercicio 4: UPDATE y ROLLBACK en MySQL

Consigna

Utilizando el motor de base de datos MySQL abra una conexión con la aplicación cliente MySQL (cliente en línea de comando) a la base de datos del ejercicio 1 de la práctica de repaso, modifique el precio de un producto en un 20 % (que sea un producto que no esté actualizando ninguno de sus compañeros), verifique que se ha cambiado el precio y luego haga un rollback de la transacción. Verifique si el cambio de precio se almacenó o no.

Idea

El experimento ilustra la propiedad de **atomicidad** (la “A” de ACID): una transacción o se aplica completa o no se aplica en absoluto. ROLLBACK es la primitiva que materializa esa propiedad cuando la transacción todavía no fue confirmada. Mientras la transacción esté abierta, las escrituras son visibles *dentro* de ella pero no son persistentes ni visibles desde afuera (en READ COMMITTED o más fuerte).

Solución

```
mysql -u root -p
```

```
USE practico1a_mysql;

SET autocommit = 0;
START TRANSACTION;

-- Producto elegido: el 104 esta libre en la carga del practico 1
SELECT cod_producto, descripcion, precio
FROM producto
WHERE cod_producto = 104
FOR UPDATE;           -- toma lock exclusivo de fila

-- Guardar el precio original para comparar
SELECT @precio_original := precio
```



```
FROM producto
WHERE cod_producto = 104;

-- Aumentar 20%
UPDATE producto
SET precio = ROUND(precio * 1.20, 2)
WHERE cod_producto = 104;

-- Verificar dentro de la transaccion: debe verse el nuevo precio
SELECT cod_producto,
       @precio_original AS precio_antes,
       precio           AS precio_despues_update
FROM producto
WHERE cod_producto = 104;

-- Deshacer
ROLLBACK;

-- Verificar despues del rollback: debe volver al precio original
SELECT cod_producto, precio AS precio_despues_rollback
FROM producto
WHERE cod_producto = 104;
```

Resultado esperado. Con los datos del Práctico 1, para `cod_producto = 104`:

- Precio inicial: 450000.00.
- Tras el UPDATE (visible solo dentro de la transacción): 540000.00.
- Tras el ROLLBACK: 450000.00 (vuelve al valor original).

Por qué funciona / por qué se rompe

- ROLLBACK usa los *undo logs* de InnoDB: cada modificación se registra en una versión paralela; al hacer rollback, el motor reinstaura las versiones originales y libera los locks tomados.
- Hasta el COMMIT, las escrituras viven en undo/redo logs y *no son visibles* desde otras sesiones (asumiendo READ COMMITTED o más fuerte). Por eso una segunda sesión consultando `cod_producto = 104` mientras la primera tiene la transacción abierta verá el valor original.
- Si la sesión se cae a mitad de camino (antes del commit), InnoDB hace rollback automático al recuperarse: misma garantía de atomicidad.

En MySQL 8+

La consigna ya pide MySQL; la *Solución* es la versión MySQL 8+. Notas adicionales:

- SET `autocommit = 0` convierte la sesión en modo transaccional. Sin esto, cada sentencia se commitea sola y ROLLBACK no tendría nada que deshacer.
- Alternativa equivalente: START TRANSACTION ya inicia una transacción explícita aunque autocommit esté en 1.
- Para confirmar que el lock se liberó: SELECT * FROM information_schema.INNODB_TRX; no debe mostrar la transacción tras el rollback.

6. Ejercicio 5: Dos sesiones MySQL en SERIALIZABLE

Consigna

Utilizando el motor de base de datos MySQL, abra dos conexiones a la base de datos del ejercicio 1 de la práctica de repaso, las dos en modo de transacción serializable, intente modificar un mismo registro de la tabla cliente por medio de las dos transacciones. ¿Qué sucede? Para finalizar realice el commit de ambas transacciones.

Idea

El nivel **SERIALIZABLE** es el más estricto del estándar SQL 92: prohíbe lecturas sucias, no repetibles y phantoms. En MySQL/InnoDB se implementa con *locks*: cada **SELECT** toma S-lock implícito, cada modificación toma X-lock. Cuando dos transacciones intentan modificar el mismo registro, la segunda queda esperando el lock que tiene la primera y la concurrencia sobre esa fila se vuelve, en efecto, serial.

Solución

Sesión A (Terminal 1):

```
USE practico1a_mysql;

SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SET autocommit = 0;
START TRANSACTION;

SELECT nro_cliente, direccion
FROM cliente
WHERE nro_cliente = 1;

UPDATE cliente
SET direccion = 'San Martin 123 - A'
WHERE nro_cliente = 1;
```

En este punto la fila `nro_cliente = 1` queda con X-lock tomado por A.

Sesión B (Terminal 2):

```
USE practico1a_mysql;

SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SET autocommit = 0;
START TRANSACTION;

SELECT nro_cliente, direccion
FROM cliente
WHERE nro_cliente = 1;    -- queda esperando (S-lock vs X-lock)

UPDATE cliente
SET direccion = 'San Martin 123 - B'
WHERE nro_cliente = 1;    -- queda esperando si paso el SELECT
```

La sentencia de B queda colgada esperando el lock de A. No hay ejecución concurrente real sobre esa fila.

Cierre pedido:

1. En Sesión A: **COMMIT;**.
2. Sesión B se desbloquea y la operación pendiente prosigue.
3. En Sesión B: **COMMIT;**.

Resultado final:

```
SELECT nro_cliente, direccion FROM cliente WHERE nro_cliente = 1;
-- 'San Martin 123 - B' (B se confirio despues de A)
```

Por qué funciona / por qué se rompe

- En **SERIALIZABLE**, InnoDB convierte todos los **SELECT** en lecturas con S-lock implícito. Si A ya tiene X-lock sobre la fila, B no puede ni siquiera leerla hasta que A confirme.
- Si la espera supera `innodb_lock_wait_timeout` (50 s por defecto), MySQL aborta la transacción con el error **1205** (*Lock wait timeout exceeded*). El default puede modificarse a nivel sesión: `SET innodb_lock_wait_timeout = 5;`.
- Si las dos transacciones leyeran filas distintas y luego se cruzaran en sentido opuesto, podría producirse *deadlock*; InnoDB lo detecta y aborta una de las dos con el error **1213**.

En MySQL 8+

La consigna ya pide MySQL. Notas adicionales sobre **SERIALIZABLE** en InnoDB:

- En **REPEATABLE READ** (el default) los **SELECT** normales se sirven desde un *snapshot* MVCC sin tomar locks, y solo **SELECT ... FOR UPDATE/FOR SHARE** toman locks explícitos. En **SERIALIZABLE**, todo **SELECT** pasa implícitamente a **LOCK IN SHARE MODE**.
- Para diagnosticar el bloqueo en tiempo real:

```
-- Que transacciones estan activas y cuanto esperaron
SELECT trx_id, trx_state, trx_started, trx_mysql_thread_id, trx_query
FROM information_schema.INNODB_TRX;

-- Que sesion bloquea a que sesion (MySQL 8+: tabla performance_schema)
SELECT * FROM performance_schema.data_locks;
SELECT * FROM performance_schema.data_lock_waits;
```

7. Ejercicio 6: Lectura no repetible

Consigna

Diseñe una prueba en MySQL donde se pueda ver que una transacción en diferentes lecturas del mismo registro devuelva diferentes valores (sin que esta transacción actualice el registro). ¿Qué niveles de aislamiento permiten esto?

Idea

La *lectura no repetible* (*non-repeatable read*) es una de las tres anomalías clásicas. Sucede cuando T_1 lee un registro, T_2 lo modifica y commitea, y T_1 lee de nuevo encontrando un valor distinto. Lo que se rompió no es la durabilidad de T_2 , sino la sensación de “mundo estable” que debería tener T_1 durante su vida.

Solución

Nivel	Lectura sucia	No repetible	Phantom
READ UNCOMMITTED	Posible	Posible	Posible
READ COMMITTED	No posible	Posible	Posible
REPEATABLE READ	No posible	No posible	Posible (raro en InnoDB)
SERIALIZABLE	No posible	No posible	No posible

Sesión A (la que solo lee):

```
USE practico1a_mysql;

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET autocommit = 0;
START TRANSACTION;

SELECT cod_producto, precio
FROM producto
WHERE cod_producto = 102;      -- supongamos 8000.00
```

Sesión B (la que modifica entre lecturas de A):

```
USE practico1a_mysql;
SET autocommit = 0;
START TRANSACTION;

UPDATE producto
SET precio = precio + 500
WHERE cod_producto = 102;

COMMIT;      -- confirma la modificacion
```

Vuelta a Sesión A: segunda lectura del mismo registro.

```
SELECT cod_producto, precio
FROM producto
WHERE cod_producto = 102;      -- ahora 8500.00

COMMIT;
```

A ve un valor distinto en dos lecturas dentro de la misma transacción sin haberlo modificado: lectura no repetible.

Niveles que lo permiten en MySQL/InnoDB:

- READ UNCOMMITTED y READ COMMITTED: lo permiten.
- REPEATABLE READ y SERIALIZABLE: no lo permiten. En REPEATABLE READ (default de InnoDB), las lecturas consistentes se sirven contra un snapshot fijo en el primer SELECT.

Verificación complementaria: repitiendo con REPEATABLE READ en la sesión A, la segunda lectura devuelve el valor original (8000.00):

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

Por qué funciona / por qué se rompe

- En READ COMMITTED, cada SELECT toma un snapshot *nuevo* que refleja todos los commits visibles en ese momento. Por eso B (que commiteó entre las dos lecturas de A) se ve.
- En REPEATABLE READ, el snapshot se establece en el primer SELECT de la transacción y se mantiene hasta el fin. InnoDB sirve todas las lecturas posteriores contra ese snapshot, ignorando commits de otras transacciones.
- En READ UNCOMMITTED, ni siquiera hace falta que B commitee: A puede llegar a ver las escrituras no confirmadas de B (lectura sucia).

En MySQL 8+

La consigna ya pide MySQL. Vale anotar que InnoDB tiene una particularidad respecto del estándar SQL: en REPEATABLE READ, gracias a los *next-key locks* (locks combinados sobre fila + gap), InnoDB *también previene phantoms* en muchos casos en los que el estándar SQL los permite. Por eso la tabla de anomalías de InnoDB es más estricta que la teórica para REPEATABLE READ.

8. Ejercicio 7: Deferrable Constraints en PostgreSQL

Consigna

Considere la siguiente base de datos implementada en postgres:

- Cliente(dni, nombre, apellido, dirección, tarifa)
- Automovil(patente, marca, modelo, dni, nro_categoria)
- Categoría(nro_categoria, tasa)
- Taller(nro_taller, nombre, dirección)
- Accidente(nro_accidente, dni, patente, código_taller, fecha, costo)

a) Considere una transacción que realice lo siguiente:

```
INSERT INTO Accidente values (1,'DNI 1', 'PAT 1', 1,'2009-10-10',234);
-- el automovil no esta registrado, se registra en la proxima instruccion
INSERT INTO Automovil values ( 'PAT 1', .....);
```

Hint: investigue Deferrable Constraints.

- b) Modifique el esquema de la base de datos de tal manera que la ejecución de la transacción anterior no genere ninguna excepción.
- c) Modifique el esquema de la base de datos de tal manera que la ejecución de la transacción anterior genere una excepción.

Idea

Las *Foreign Keys* en SQL estándar pueden chequearse de dos maneras: *inmediata* (después de cada sentencia, default) o *diferida* (hasta el COMMIT). La transacción del enunciado tiene una inconsistencia transitoria (insertar un accidente que referencia un automóvil aún no cargado), por lo que solo funciona si la FK es DEFERRABLE INITIALLY DEFERRED.

Solución

a) *Comportamiento con esquema normal (FK inmediata)*. El primer INSERT falla:

```
BEGIN;

INSERT INTO practico1c.accidente
(nro_accidente, dni, patente, nro_taller, fecha, costo)
VALUES
(1, 30111222, 'PAT1', 1, DATE '2009-10-10', 234);
-- ERROR: insert or update on table "accidente" violates foreign key
-- constraint "fk_accidente_automovil"

INSERT INTO practico1c.automovil
(patente, marca, modelo, dni, nro_categoria)
VALUES ('PAT1', 'FIAT', 2010, 30111222, 1);

COMMIT;      -- ya no aplica: la transaccion esta abortada
```

b) *Esquema que evita la excepción*. Hacer la FK deferible y diferirla por defecto:

```
ALTER TABLE practico1c.accidente
  ALTER CONSTRAINT fk_accidente_automovil
  DEFERRABLE INITIALLY DEFERRED;
```

Con eso la transacción funciona tal cual:

```
BEGIN;

-- (Opcional, ya esta en DEFERRED por la ALTER)
SET CONSTRAINTS fk_accidente_automovil DEFERRED;

INSERT INTO practico1c.accidente
  (nro_accidente, dni, patente, nro_taller, fecha, costo)
VALUES (1, 30111222, 'PAT1', 1, DATE '2009-10-10', 234);

INSERT INTO practico1c.automovil
  (patente, marca, modelo, dni, nro_categoria)
VALUES ('PAT1', 'FIAT', 2010, 30111222, 1);

COMMIT;      -- recién aquí se valida la FK; en este momento PAT1 existe
```

c) Esquema que fuerza excepción. Volver la FK a no deferible:

```
ALTER TABLE practico1c.accidente
  ALTER CONSTRAINT fk_accidente_automovil
  NOT DEFERRABLE;
```

La misma transacción vuelve a fallar exactamente como en (a).

Consulta de control:

```
SELECT conname, condeferrable, condeferred
FROM pg_constraint
WHERE conrelid = 'practico1c.accidente'::regclass
  AND conname = 'fk_accidente_automovil';
-- condeferrable=t, condeferred=t -> caso (b)
-- condeferrable=f               -> caso (c)
```

Por qué funciona / por qué se rompe

- Con FK inmediata, PostgreSQL ejecuta el chequeo al final de cada INSERT: el primero falla y la transacción queda abortada (se ignoran las sentencias siguientes hasta el COMMIT, que se convierte en ROLLBACK implícito).
- Con DEFERRABLE INITIALLY DEFERRED, el chequeo se posterga hasta el COMMIT: el orden de los INSERT dentro de la transacción no importa siempre que el estado final cumpla todas las constraints.
- Las constraints diferidas son la herramienta canónica para insertar ciclos de referencias (A apunta a B y B apunta a A) o cargar bulk de datos de tablas relacionadas sin orden topológico.

En MySQL 8+

InnoDB no soporta foreign keys deferibles. Todas las FKs se chequean inmediatamente después de cada sentencia. Esta es una limitación bien conocida y documentada del motor. Hay dos workarounds:

- **Reordenar los INSERT:** insertar primero en `automovil` y después en `accidente`. Es la solución preferida si se controla el código.
- **Desactivar temporalmente el chequeo de FKs:** `SET FOREIGN_KEY_CHECKS = 0;` antes de los inserts y `SET FOREIGN_KEY_CHECKS = 1;` después. **No es equivalente** a `DEFERRED`: si al final hay inconsistencias, MySQL no las detecta (a diferencia de PostgreSQL que valida al `COMMIT`). Útil solo para cargas controladas (importación de dumps).

```
-- Workaround MySQL (NO equivale a DEFERRED en PostgreSQL):
SET FOREIGN_KEY_CHECKS = 0;
START TRANSACTION;
INSERT INTO accidente VALUES (1, 'DNI1', 'PAT1', 1, '2009-10-10', 234);
INSERT INTO automovil VALUES ('PAT1', 'FIAT', 2010, ...);
COMMIT;
SET FOREIGN_KEY_CHECKS = 1;

-- IMPORTANTE: si tras el SET FOREIGN_KEY_CHECKS=1 quedan inconsistencias,
-- MySQL las acepta. PostgreSQL en cambio aborta el COMMIT.
```

Diferencia clave con PostgreSQL/Oracle/SQL estándar: la cláusula `DEFERRABLE INITIALLY DEFERRED` es estándar SQL y PostgreSQL/Oracle la soportan; MySQL la *ignora silenciosamente* si se la incluye en `CREATE TABLE`. La parte (b) del ejercicio se puede resolver de forma idiomática solo en motores que respeten el estándar.

9. Ejercicio 8: MVCC en PostgreSQL (`xmin` / `xmax`)

Consigna

Utilizando postgres sobre la base de datos definida en el ejercicio b) de la práctica de SQL, diseñe y ejecute una prueba donde dos transacciones vean diferentes valores para la misma fila. Observe los valores `xmin` y `xmax`.

Idea

PostgreSQL implementa *MVCC* (Multi-Version Concurrency Control): cada `UPDATE` no machaca la versión vieja sino que crea una versión nueva. Cada tupla tiene dos columnas de sistema: `xmin` (XID de la transacción que la insertó) y `xmax` (XID de la transacción que la invalida; 0 si está viva). Las lecturas no toman lock: el snapshot decide cuál de las versiones ver, permitiendo que dos transacciones concurrentes vean valores distintos para la misma fila sin esperarse.

Solución

Sesión A — transacción T_A :

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT txid_current() AS tx_a;

SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';

UPDATE vehiculo
SET saldo_actual = saldo_actual + 100
WHERE patente = 'AA123BB';

SELECT patente, saldo_actual, xmin, xmax
```

```
FROM vehiculo
WHERE patente = 'AA123BB';

-- NO HACER COMMIT TODAVIA
```

Dentro de T_A : antes del UPDATE se ve la versión original (con su $xmin$ y $xmax = 0$); después del UPDATE, T_A ve la nueva versión ($xmin = tx_a$, $xmax = 0$). La versión vieja queda con $xmax = tx_a$ pero T_A ya no la ve.

Sesión B — transacción T_B , en paralelo, antes del commit de A:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT txid_current() AS tx_b;

SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';
```

Dentro de T_B : ve el valor *anterior* (porque el snapshot de B no incluye los cambios no confirmados de A). En esa versión, $xmin$ corresponde a la transacción que la creó; $xmax$ puede mostrar tx_a (que está invalidándola), pero para B sigue siendo la versión visible.

Aquí está la observación pedida: T_A y T_B ven valores distintos para la misma fila, simultáneamente y sin que ninguna espere.

Cierre y segunda lectura:

```
-- Sesión A
COMMIT;

-- Sesión B (sigue en su transacción READ COMMITTED)
SELECT patente, saldo_actual, xmin, xmax
FROM vehiculo
WHERE patente = 'AA123BB';

COMMIT;
```

En READ COMMITTED, esta segunda lectura de B toma un snapshot nuevo y ya ve el valor confirmado por A.

Interpretación de $xmin/xmax$:

- Una versión viva tiene $xmax = 0$.
- Un UPDATE produce dos versiones: la vieja queda con $xmax = XID$ del UPDATE; la nueva con $xmin = XID$ del UPDATE, $xmax = 0$.
- Un DELETE marca la fila con $xmax = XID$ del DELETE sin crear nueva versión.
- Las versiones muertas las elimina VACUUM (o autovacuum).

Por qué funciona / por qué se rompe

- MVCC permite *lecturas sin locks*: B no tiene que esperar a A para leer porque cada transacción ve la última versión visible para su snapshot, no la última versión absoluta.
- El precio es espacio: las versiones muertas se acumulan hasta que el autovacuum las recoge; en cargas con muchos UPDATE esto puede generar *bloat*.

- Si B intenta modificar la misma fila que A está modificando, igual aparece un lock: el lock exclusivo solo aplica a escrituras, las lecturas siempre van por MVCC.
- En REPEATABLE READ/SERIALIZABLE, el snapshot se fija al primer SELECT de la transacción y se mantiene; un intento de modificar una fila ya modificada por otra transacción confirmada produce el error “could not serialize access due to concurrent update”.

En MySQL 8+

InnoDB *también* implementa MVCC, pero **no expone columnas xmin/xmax** en las tuplas: ambas viven en el undo log interno y no son consultables desde SQL. Lo que sí se puede observar es el comportamiento equivalente y algunos metadatos sobre las transacciones activas:

```
-- 1) Reproducir el escenario de dos sesiones viendo valores distintos
-- (default de InnoDB es REPEATABLE READ; con READ COMMITTED para emular pg):

-- Sesión A
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
SELECT patente, saldo_actual FROM vehiculo WHERE patente='AA123BB';
UPDATE vehiculo SET saldo_actual = saldo_actual + 100 WHERE patente='AA123BB';
-- no commit todavía

-- Sesión B (en paralelo)
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
SELECT patente, saldo_actual FROM vehiculo WHERE patente='AA123BB';
-- B ve el valor previo: MVCC le sirve la versión anterior desde undo log

-- 2) Inspeccionar el lado interno: que transacciones están vivas y que están haciendo
SELECT trx_id, trx_state, trx_started, trx_isolation_level, trx_query
FROM information_schema.INNODB_TRX;

-- 3) Ver locks tomados (MySQL 8+):
SELECT * FROM performance_schema.data_locks;
```

Diferencias relevantes con PostgreSQL:

- PostgreSQL guarda las versiones *en la tabla* (cada fila ocupa lugar hasta que VACUUM la limpia); InnoDB guarda las versiones *en el undo log* de tablespace separado.
- xmin/xmax son columnas pseudo-públicas en PostgreSQL; en InnoDB el equivalente (DB_TRX_ID, DB_ROLL_PTR) no se puede consultar desde SQL.
- En InnoDB no hay *bloat*: las versiones muertas se reciclan automáticamente al cerrarse las transacciones que las pudieran necesitar. Por eso InnoDB no necesita un VACUUM explícito (lo hace el *purge thread*).

Conclusiones

El práctico recorre los dos planos en los que se piensa la concurrencia de transacciones:

1. **Plano teórico** (Ejercicios 1–3): cómo se verifica que un schedule es “equivalente a una ejecución serial” aunque haya intercalado real. La herramienta es el grafo de precedencia (test de serializabilidad por conflicto), y los dos protocolos que garantizan schedules serializables son el bloqueo de dos fases y la ordenación por marcas temporales.
2. **Plano práctico** (Ejercicios 4–8): cómo se materializan estos conceptos en motores reales. ROLLBACK aplica atomicidad. Los niveles de aislamiento permiten relajar la serializabilidad

estricta para ganar concurrencia, asumiendo determinadas anomalías. Las constraints diferidas permiten transacciones con inconsistencias intermedias. El MVCC de PostgreSQL muestra cómo distintos snapshots permiten a varias transacciones ver versiones distintas de la misma fila sin bloquearse.

La idea común detrás del práctico es: *las transacciones aíslan al programador del caos de la concurrencia, pero ese aislamiento tiene costos*. Los protocolos y los niveles de aislamiento son los manijos que el sistema ofrece para regular ese trade-off entre consistencia y rendimiento. Una nota cruzada que aparece en varios ejercicios: **InnoDB no expone los mecanismos teóricos directamente**: no permite 2PL puro (es siempre estricto), no usa marcas temporales (usa 2PL + detección de deadlocks), no soporta FKs deferibles (las chequea inmediatamente) y oculta `xmin/xmax` en el undo log. Cuando la consigna pide reproducir conceptos teóricos en MySQL, en muchos casos hay que conformarse con observar el *efecto* sin acceso directo al mecanismo.